PolicyGPT

A Project report submitted in fulfillment of the requirements for the completion of the

INFOSYS SPRINGBOARD

Virtual Internship Program 7.0

On

Government Policy & Public Scheme Intelligence Platform

By

Team Members:

1. Elahiya
2. Priya Darshini
3. Suresh Kumar
4. Spurthi
5. Sri Charan

Team No.: 3

Domain:

Angular Full Stack Development

Under the Guidance of

Infosys SpringBoard Mentor

Anuradha

Academic Year:

2026–2027

DECLARATION BY THE TEAM

We, **Elahiya, Priya Darshini, Suresh Kumar, Spurthi, and Sri Charan**, members of **Team 3**, hereby declare that the project titled **"PolicyGPT: Government Policy & Public Scheme Intelligence Platform"** has been developed and documented by us as part of the **Infosys Springboard Virtual Internship Program**.

We declare that the work presented in this project report is based on our team's development, implementation, testing, and documentation activities carried out during the internship. The project has been developed using the technologies and tools specified for the project, including Angular, TypeScript, Python, FastAPI, and PostgreSQL.

We further declare that the information, implementation details, diagrams, screenshots, results, and other materials presented in this report are related to the **PolicyGPT project developed by Team 3** and have been prepared for submission as part of the **Infosys Springboard Virtual Internship Program**.

The project report has been prepared to document the requirements, system design, architecture, modules, implementation, database design, testing, deployment, results, and future enhancements of the PolicyGPT platform.

Team Members

Elahiya
Priya Darshini
Suresh Kumar
Spurthi
Sri Charan

Team No.: 3

ACKNOWLEDGEMENT

This internship provided us with an opportunity to gain practical experience in software development and apply our technical knowledge to a real-world project. Throughout the internship, we were able to enhance our understanding of Angular, TypeScript, Python, FastAPI, PostgreSQL, REST APIs, authentication, database management, testing, and full-stack application development**.**

We sincerely thank the **Infosys Springboard team and our project mentor** for providing valuable guidance, feedback, and support throughout the different stages of the project. Their guidance helped us understand the project requirements, plan the implementation, overcome technical challenges, and complete the assigned milestones successfully.

We would also like to thank our **Team 3 members — Elahiya, Priya Darshini, Suresh Kumar, Spurthi, and Sri Charan** — for their cooperation, coordination, and individual contributions throughout the development of the PolicyGPT platform.

Finally, we would like to thank everyone who supported and encouraged us directly or indirectly throughout the completion of this internship project. The experience gained through the **Infosys Springboard Virtual Internship Program** has helped us strengthen our technical skills, teamwork, problem-solving abilities, and understanding of the software development process.

— Team 3

ABSTRACT

PolicyGPT: Government Policy & Public Scheme Intelligence Platform is a full-stack web application developed as part of the **Infosys Springboard Virtual Internship Program**. The project is designed to provide a centralized platform for accessing, searching, analyzing, and understanding government policies, public welfare schemes, notifications, circulars, and other official information. It aims to make government-related information easier to discover and understand for citizens and other stakeholders.

The platform brings several important functionalities together in a single system, including:

- **Policy and Scheme Management**
- **Policy Search and Filtering**
- **Eligibility Checking and Scheme Recommendations**
- **Policy and Scheme Comparison**
- **Notifications, Dashboards and Analytics**
- **Reports, Export and Feedback Support**

The **Eligibility Checker** allows users to provide details such as age, gender, income, occupation, education, location, social category, and disability status to identify relevant government schemes. The system also supports different user roles, including citizens, government officials, researchers, organizations, and guest users.

The application is developed using **Angular and TypeScript** for the frontend, **Python and FastAPI** for the backend, and **PostgreSQL** for database management. Security and access control are supported through **JWT Authentication, OAuth2, and Role-Based Access Control**. The project follows a milestone-based development process covering requirements and design, module development, analytics and notifications, reporting, testing, deployment, and documentation.

Overall, PolicyGPT aims to improve accessibility, awareness, and ease of use of government policies and welfare schemes by bringing essential information and services together through one integrated digital platform.

TABLE OF CONTENTS

Chapter 5 Technology Stack

Chapter 6 Modules and Implementation

Chapter 7 Database Design

Chapter 8 Testing and Validation

Chapter 9 Milestone-wise Development

Chapter 10 Deployment and Results

Chapter 11 Conclusion and Future Enhancements

CHAPTER 1
INTRODUCTION

1.1 Project Overview and Background

Government policies and public welfare schemes provide important benefits and services to citizens, but related information is often spread across different sources, making it difficult to find and understand.

PolicyGPT: Government Policy & Public Scheme Intelligence Platform is a full-stack web application developed to provide a **centralized platform** for accessing, searching, analyzing, and understanding government policies, public welfare schemes, regulations, notifications, circulars, and related information.

The platform supports **citizens, government officials, researchers, students, and organizations** with features such as policy and scheme management, search and filtering, eligibility checking, comparison, notifications, dashboards, analytics, reports, and feedback support. It also provides role-based access for **Administrators, Government Officials, Citizens, Researchers, Organizations, and Guest Users**.

The application uses **Angular and TypeScript** for the frontend, **Python and FastAPI** for the backend, and **PostgreSQL** for database management. The development is organized into four milestones covering **requirements and design, core module development, analytics and notifications, and testing, deployment, and documentation**.

1.2Problem Statement and Motivation

Government policies and public welfare schemes provide important services and benefits to citizens, but related information is often available through different

government departments, websites, notifications, circulars, and official documents. This makes it difficult for users to quickly find, understand, compare, and track information relevant to their needs.

The major challenges addressed by the **PolicyGPT** project are:

- Difficulty in **finding relevant government policies and schemes**.
- Lack of a **centralized platform** for policy and scheme information.
- Difficulty in understanding **scheme eligibility requirements**.
- Challenges in **comparing different policies and schemes**.
- Difficulty in tracking **policy updates, scheme updates, deadlines, and application notifications**.
- Limited access to organized **dashboards, analytics, and reports** for monitoring government policies and schemes.

The motivation behind PolicyGPT is to address these challenges by bringing important government information and related services together in a single platform. The system provides **policy and scheme search, eligibility checking, comparison, notifications, dashboards, analytics, reporting, and feedback support**. It also supports different user roles, including citizens, government officials, researchers, students, organizations, and administrators.

Overall, PolicyGPT aims to provide a **centralized, organized, accessible, and user-friendly digital platform** that makes government policy and public scheme information easier to discover, understand, compare, and track.

1.3 Objectives

The main objective of **PolicyGPT** is to develop a centralized full-stack platform that makes government policies and public welfare schemes easier to access, search, understand, and manage. The specific objectives of the project are:

1. To provide a **centralized platform** for accessing government policies, public welfare schemes, notifications, circulars, and related information.
2. To implement **secure user authentication and role-based access control** for different categories of users.
3. To develop **Policy Management and Public Scheme Management** functionalities for organizing and maintaining policy and scheme information.
4. To provide **keyword-based and advanced search and filtering** based on categories, states, ministries, departments, publication date, and status.
5. To develop an **Eligibility Checker** that helps users identify suitable schemes based on parameters such as age, gender, income, occupation, education, location, social category, and disability status.
6. To provide **policy and scheme comparison** based on benefits, eligibility, departments, and application processes.
7. To implement a **Notification System** for new policy alerts, scheme updates, deadline reminders, and application notifications.
8. To provide **dashboards and analytics** for citizens, government officials, and administrators.
9. To generate **policy, scheme, user activity, department, and analytics reports**, with PDF and Excel export capabilities.
10. To provide **feedback and support facilities** for users to report issues, access FAQs, and seek assistance.
11. To complete the project through **testing, issue fixing, deployment, documentation, and final demonstration** as part of the project milestones.

1.4 Scope of the Project and Target Users

The scope of **PolicyGPT** covers the development of a centralized full-stack platform for accessing, searching, managing, and understanding government policies and public welfare schemes. The system includes **user authentication and role management, policy and scheme management, policy search and**

filtering, eligibility checking, policy and scheme comparison, notification management, dashboards and analytics, reports and export, and feedback and support. It allows users to search for relevant policies and schemes, check eligibility based on their profile details, compare available options, and receive important updates and notifications. The platform also provides role-based functionalities for citizens, government officials, researchers, organizations, administrators, and guest users. The project scope extends through the complete development cycle, including requirements and UI design, database design, backend and frontend development, testing, deployment, documentation, and final project demonstration.

Target Users

S. No.	Target User	Purpose / Major Activities
1	**Citizen**	Search government policies and schemes, check eligibility, view recommendations, and receive notifications.
2	**Government Official**	Manage policies and schemes, monitor usage, and access analytics and reports.
3	**Researcher**	Search, explore, and analyze government policies and public schemes for research purposes.
4	**Student**	Access government policies and schemes for academic learning and research.
5	**Organization**	Access relevant government policies, schemes, and related information for organizational requirements.
6	**Administrator**	Manage users, policies, analytics, reports, and audit-related information.
7	**Guest User**	Access permitted public government information without full registered-user privileges.

1.5 Benefits of the Proposed System

PolicyGPT provides several benefits by bringing government policies, public welfare schemes, and related services together on a single platform. It reduces the effort required to search across different sources and helps users access relevant information in an organized manner. The **search and filtering** features make it easier to find policies and schemes, while the **Eligibility Checker** helps users identify schemes based on their personal and eligibility details. The **comparison feature** allows users to examine different policies and schemes, while the **notification system** helps users stay informed about important updates, deadlines, and application-related information. The platform also provides **dashboards, analytics, and reports** to support government officials and administrators in monitoring policy and scheme information. Overall, PolicyGPT aims to improve the **accessibility, awareness, organization, and ease of use of government information** through an integrated digital platform.

- **Centralized Information Access:**
  Provides government policies, welfare schemes, notifications, and related information through a single platform.
- **Easy Search and Filtering:**
  Allows users to quickly find relevant policies and schemes using keywords and different filters.
- **Eligibility Assistance:**
  Helps users determine suitable government schemes based on their personal and eligibility details.
- **Policy and Scheme Comparison:**
  Enables users to compare different policies and schemes based on important information such as benefits, eligibility, and application details.

- **Timely Notifications:**

  Keeps users informed about new policies, scheme updates, deadlines, and application-related notifications.

- **Role-Based Access:**

  Provides appropriate access and functionalities for citizens, government officials, researchers, students, organizations, administrators, and guest users.

- **Dashboards and Analytics:**

  Helps government officials and administrators monitor policy, scheme, user, and system-related information through dashboards and analytics.

- **Reports and Support:**

  Supports report generation and provides feedback and support facilities for better management and user assistance.

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

A literature survey provides an overview of existing systems, approaches, and technologies related to the development of PolicyGPT. Government information systems have increasingly moved toward digital platforms that allow users to access policies, welfare schemes, public services, and official information online. However, information may still be distributed across different sources, making it difficult for users to discover relevant policies, understand scheme eligibility, compare available options, and stay informed about updates.

PolicyGPT is designed around these requirements by bringing policy and scheme management, search, eligibility checking, comparison, notifications, dashboards, analytics, and reporting into a single platform. The project specification identifies centralized access, intelligent search, eligibility checking, comparison, policy tracking, notification management, and analytics as important capabilities of the proposed platform.

This chapter reviews the major areas related to the proposed system and identifies the gap that PolicyGPT aims to address.

2.2 Existing Government and Digital Information Systems

Government information systems are developed to provide citizens and other stakeholders with access to government policies, public welfare schemes, services, notifications, circulars, and official documents through digital platforms. These systems help improve the availability of government information and reduce dependence on traditional manual processes.

Digital government services make it easier for users to access information online. However, information related to different policies and schemes may be available through different sources, requiring users to search across multiple platforms. This can make it difficult to quickly find relevant information and understand the complete details of a scheme or policy.

PolicyGPT addresses this requirement by providing a **centralized digital platform** where users can access, search, analyze, and understand government policies and public welfare schemes along with related information. The platform is intended to support citizens, government officials, researchers, students, and organizations.

2.3 Policy, Scheme, Search and Eligibility Systems

Policy and scheme management systems help organize government policies and public welfare schemes by maintaining their details, categories, eligibility rules, updates, and related information. Such systems make policy and scheme information easier to manage and access.

Search functionality helps users find relevant policies and schemes using keywords and filters such as category, state, ministry, and department. Eligibility-based systems further assist users by analyzing their details and identifying schemes that may be suitable for them.

PolicyGPT combines these functionalities in one platform. It provides **policy and scheme management, search and filtering, eligibility checking, and scheme recommendations**, helping users find relevant government information more efficiently.

2.4 Notification and Tracking Systems

Notification and tracking systems help users stay informed about important changes and activities related to government policies and welfare schemes. They can

provide alerts about new policies, scheme updates, application status, deadlines, and other important information.

PolicyGPT includes a **Notification Module** that supports alerts and reminders for new policies, scheme updates, deadlines, and applications. The platform also provides notification channels such as **email, SMS, and in-app notifications**, helping users receive timely information.

These features improve awareness and help users track important policy and scheme-related information without repeatedly searching for updates.

2.5 Limitations and Research Gap

Existing government information systems provide access to policies, schemes, and public services, but users may still face difficulties in finding, understanding, comparing, and tracking relevant information. The major limitations and the corresponding research gap are summarized below.

S. No.	Limitations of Existing Approaches	Research Gap / Need
1	Information may be distributed across different sources.	A centralized platform is needed.
2	Finding relevant policies and schemes can be time-consuming.	Integrated search and filtering is required.
3	Users may have difficulty identifying eligible schemes.	An eligibility checking and recommendation feature is needed.
4	Comparing different policies or schemes may require manual effort.	An integrated comparison facility is required.
5	Users may miss important updates and deadlines.	Timely notification and tracking features are needed.
6	Information management and monitoring may be fragmented.	Integrated dashboards, analytics, and reporting are needed.

PolicyGPT addresses these gaps by integrating **policy and scheme management, search, eligibility checking, comparison, notifications, analytics, and reporting** into a centralized platform.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Project Background and Existing System

Government policies and public welfare schemes provide important services and benefits to citizens. However, information related to policies, schemes, eligibility conditions, notifications, and application details may be available through different sources. This can make it difficult for users to quickly find and understand relevant information.

The existing approach generally requires users to search different sources for policy and scheme information and manually identify relevant details. Users may also need separate processes to check eligibility, compare schemes, and track important updates.

PolicyGPT was proposed to address these challenges through a centralized digital platform that brings government policy and public scheme information together with features such as search, eligibility checking, comparison, notifications, dashboards, and reporting

3.2 Limitations of Existing System

The existing approach for accessing government policies and public welfare schemes has several limitations. Since information may be available across different sources, users may face difficulty in finding and understanding the information they need.

The major limitations are:

- **Distributed Information:** Policy and scheme information may be available across multiple sources.

- **Time-Consuming Search:** Users may need to search different sources to find relevant information.
- **Limited Eligibility Assistance:** Identifying suitable schemes based on individual details may require manual effort.
- **Difficult Comparison:** Comparing benefits, eligibility, and application details of different schemes may be difficult.
- **Limited Update Tracking:** Users may miss important policy updates, scheme changes, deadlines, or application notifications.
- **Lack of Integrated Services:** Search, eligibility, comparison, notifications, analytics, and reporting may not be available through a single platform.

3.3 Proposed System and Advantages

The proposed **PolicyGPT** is a centralized full-stack platform developed to provide easy access to government policies and public welfare schemes. It integrates policy and scheme management, search, eligibility checking, comparison, notifications, dashboards, analytics, reporting, and feedback services in a single system.

Major Advantages

1. **Centralized Information:** Provides policies and schemes through a single platform.
2. **Easy Search:** Helps users find relevant information using keywords and filters.
3. **Eligibility Assistance:** Helps users identify suitable schemes based on their details.
4. **Policy and Scheme Comparison:** Allows users to compare important scheme and policy information.
5. **Timely Notifications:** Provides updates about schemes, policies, deadlines, and applications.
6. **Role-Based Access:** Provides different functionalities according to user roles.

7. **Analytics and Reports:** Supports monitoring and reporting through dashboards and reports.
8. **Improved Accessibility:** Makes government information more organized and easier to access.

3.4 Functional Requirements

Functional requirements define the major functions and services that the PolicyGPT system should provide to its users. The main functional requirements are:

1. **User Authentication and Role Management:** User registration, secure login, profile management, password reset, and role-based access.
2. **Policy Management:** Authorized users can upload, categorize, edit, approve, search, and archive government policies.
3. **Scheme Management:** Manage scheme details, categories, eligibility rules, updates, and archived schemes.
4. **Policy Search:** Search policies and schemes using keywords and filters such as category, state, ministry, and department.
5. **Eligibility Checker:** Analyze user details and identify suitable government schemes based on eligibility conditions.
6. **Policy Comparison:** Compare policies and schemes based on benefits, eligibility, departments, and application processes.
7. **Notification Management:** Provide alerts and reminders for new policies, scheme updates, deadlines, and applications.
8. **Dashboard and Analytics:** Display relevant statistics, activities, notifications, and reports for different user roles.
9. **Reports and Export:** Generate policy, scheme, user activity, department, and analytics reports with export options.
10. **Feedback and Support:** Allow users to submit feedback, report issues, and access support services.

3.5 Non-Functional Requirements

Non-functional requirements define the quality, performance, security, and usability characteristics expected from the PolicyGPT system.

1. **Security:** The system should protect user information through secure authentication and role-based access control.
2. **Performance:** The application should provide quick responses for searches, eligibility checks, and other user requests.
3. **Usability:** The interface should be simple, clear, and easy to use for different types of users.
4. **Reliability:** The system should function consistently and provide accurate responses for supported operations.
5. **Scalability:** The system should be capable of supporting additional users, policies, schemes, and features in the future.
6. **Maintainability:** The application should have a structured architecture that makes updates and maintenance easier.
7. **Availability:** The platform should be accessible to users whenever the application services are running and available.
8. **Compatibility:** The application should work effectively with the selected frontend, backend, database, and supporting technologies.

3.6 Assumptions, Dependencies and Constraints

The development of PolicyGPT is based on certain assumptions, dependencies, and constraints that may affect the system and its implementation.

Assumptions

1. Users have access to a device and internet connection to use the platform.
2. Users provide correct and relevant information when using features such as the Eligibility Checker.
3. Government policy and scheme information provided to the system is assumed to be available and maintained appropriately.

Dependencies

1. The system depends on the **Angular frontend** and **FastAPI backend** for application functionality.
2. **PostgreSQL** is used for storing and managing application data.

3. Authentication and notification services depend on their respective configurations and supporting services.
4. Development and testing depend on tools such as Git, GitHub, Postman, and Docker.

Constraints

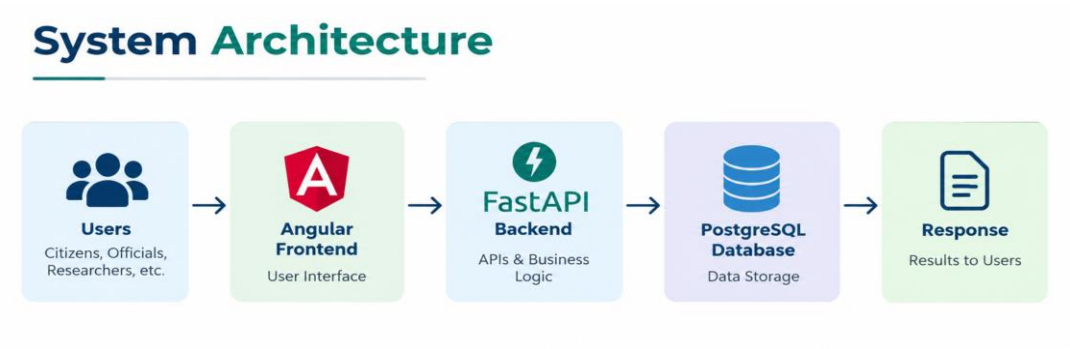
1. The accuracy of eligibility results depends on the information and eligibility rules available in the system.
2. Notification delivery depends on the availability and configuration of external notification services.
3. The system requires proper authentication and authorization to protect user and administrative functions.
4. The project scope and implementation are limited to the features defined in the project requirements.

CHAPTER 4
SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

PolicyGPT follows a **full-stack architecture** consisting of a frontend, backend, database, and supporting services. The architecture separates the user interface, application logic, and data management layers, allowing the different components to work together efficiently.

The **Angular and TypeScript frontend** provides the user interface through which users interact with the system. The **FastAPI backend**, developed using Python, handles API requests, application logic, authentication, and communication with the database. **PostgreSQL** is used to store and manage application data such as users, policies, schemes, eligibility rules, notifications, feedback, and related records.



Supporting services such as authentication and notification services are integrated with the backend according to the requirements of the application. This architecture provides a structured foundation for implementing the different PolicyGPT modules.

4.2 Architecture Components and Workflow

The PolicyGPT system consists of several interconnected components that work together to provide government policy and public scheme services. The major components are:

1. **Frontend Layer:**
   The Angular-based frontend provides the user interface for accessing different modules and services.
2. **Backend Layer:**
   The FastAPI backend handles API requests, application logic, authentication, and communication between the frontend and database.
3. **Database Layer:**
   PostgreSQL stores users, policies, schemes, eligibility rules, notifications, feedback, and other application data.
4. **Authentication and Authorization:**
   JWT/OAuth2-based authentication and role-based access control help manage secure access to system features.
5. **Supporting Services:**
   Email, SMS, and other notification services support communication with users.

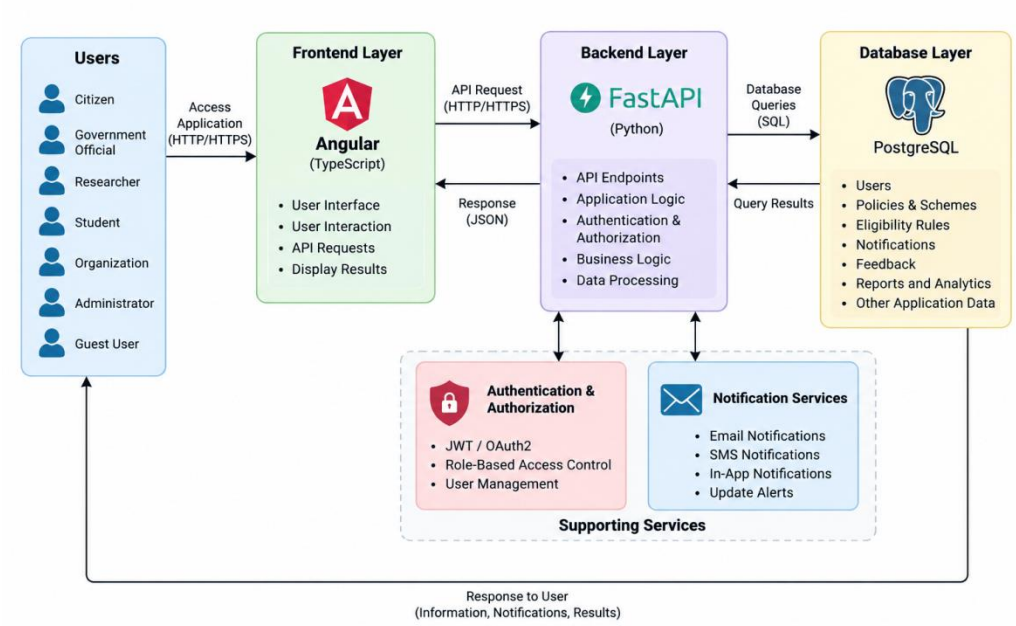


Figure 4.2: Architecture Components and System Workflow of PolicyGPT

4.3 User Roles and Access

PolicyGPT supports different types of users, with access and functionalities provided according to their roles. This role-based approach helps ensure that users can access the features relevant to their requirements.

S. No.	User Role	Main Access / Activities
1	**Citizen**	Search policies and schemes, check eligibility, view recommendations, and receive notifications.
2	**Government Official**	Manage policies and schemes, monitor information, and access analytics and reports.
3	**Researcher**	Search, explore, and analyze government policies and schemes for research.
4	**Student**	Access government policies and schemes for academic learning and research.
5	**Organization**	Access relevant policies, schemes, and related government information.
6	**Administrator**	Manage users, policies, analytics, reports, and administrative information.
7	**Guest User**	Access permitted public government information without registered-user privileges.

4.4 UML and Data Flow Diagrams

UML and Data Flow Diagrams are used to represent the structure, interactions, and flow of information within the PolicyGPT system. They provide a clear visual representation of how users interact with different system functions and how data moves between the frontend, backend, and database.

UML Diagrams

The UML diagrams represent the major actors and functions of PolicyGPT. The main actors include **Citizen, Government Official, Researcher, Student, Organization, Administrator, and Guest User**. Depending on their roles, users can access functions such as policy and sche

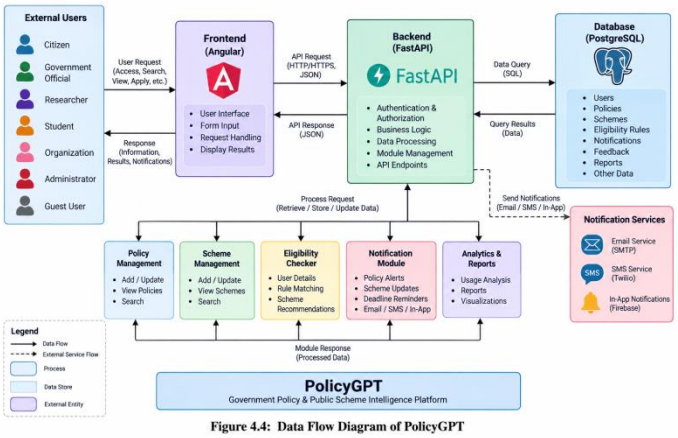


Figure 4.4: Data Flow Diagram of PolicyGPT

Data Flow Diagram

The Data Flow Diagram represents the movement of data through the system. Users send requests through the Angular frontend, which communicates with the FastAPI backend. The backend processes the requests and interacts with the PostgreSQL database to store or retrieve the required information. The processed results are then returned to the user interface.

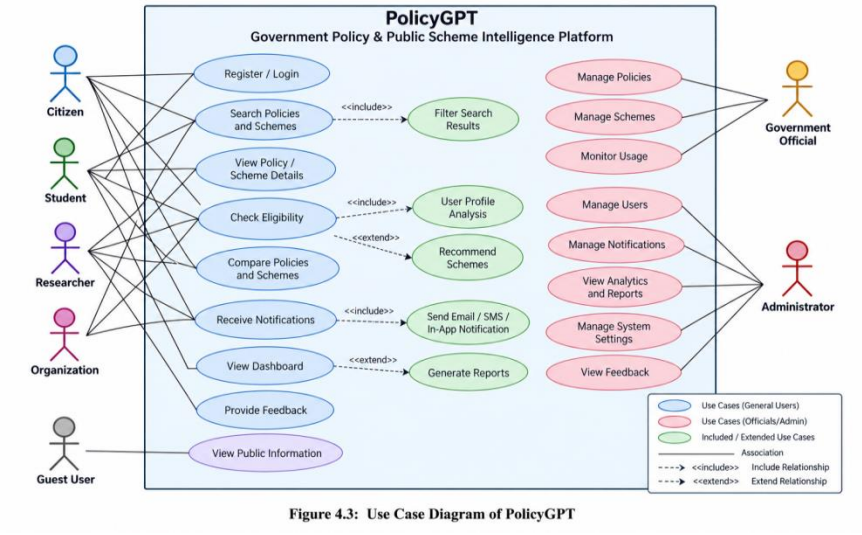


Figure 4.3: Use Case Diagram of PolicyGPT

4.5 Deployment Architecture

PolicyGPT is designed as a full-stack application with separate frontend, backend, and database components. The **Angular frontend** communicates with the **FastAPI backend**, which processes requests and interacts with the **PostgreSQL database**. Docker is included to support containerized deployment.

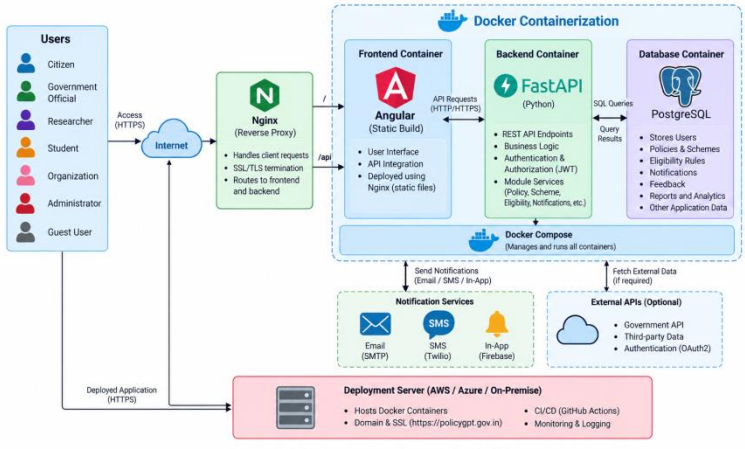


Figure 4.5: Deployment Architecture of PolicyGPT

The architecture can be deployed using containerized services, allowing the frontend, backend, and database components to be managed separately.

CHAPTER 5
TECHNOLOGY STACK

5.1 Introduction

PolicyGPT is developed using a combination of **frontend, backend, database, authentication, security, and development technologies**. These technologies work together to provide a reliable and user-friendly full-stack platform for managing and accessing government policies and public welfare schemes.

The frontend is developed using **Angular and TypeScript**, while **Python and FastAPI** are used for backend development and API services. **PostgreSQL** is used for database management, with **JWT/OAuth2** supporting authentication and role-based access. Development and deployment activities use tools such as **VS Code, Git, GitHub, Postman, Figma, and Docker**.

The following sections describe the major technologies and tools used in the development of PolicyGPT.

5.2 Frontend and Backend Technologies

PolicyGPT uses **Angular and TypeScript** for frontend development and **Python with FastAPI** for backend development. These technologies work together to provide the user interface, application logic, and communication between the frontend and database.

Frontend Technologies

- **Angular:** Used to develop the web-based user interface and application components.
- **TypeScript:** Used for implementing frontend logic and functionality.
- **Angular Material:** Used to support the design and development of the user interface.

Backend Technologies

- **Python:** Used as the primary backend programming language.

- **FastAPI:** Used to develop REST APIs and handle communication between the frontend and backend.
- **Uvicorn:** Used as the application server for running the FastAPI backend

5.3 Database, Authentication and Security

PolicyGPT uses **PostgreSQL** as the primary database for storing and managing application data. The database supports information related to users, policies, schemes, eligibility rules, notifications, feedback, reports, and other system records.

Database

- **PostgreSQL:** Used as the primary database for storing and retrieving application data.

Authentication and Security

- **JWT Authentication:** Used to authenticate users and provide secure access to protected resources.
- **OAuth2:** Used as an authentication and authorization mechanism.
- **Role-Based Access Control (RBAC):** Controls access to system functionalities based on user roles such as Citizen, Government Official, and Administrator.

This approach helps provide **secure, role-specific access** to the PolicyGPT platform.

5.4 Development, Testing and DevOps Tools

Several development and supporting tools were used during the development and testing of PolicyGPT. These tools helped the team in coding, version control, API testing, UI design, and application deployment.

- **Visual Studio Code (VS Code):** Used as the primary code editor for frontend and backend development.
- **GitHub:** Used for source-code management, branch management, and team collaboration.
- **Figma:** Used for designing UI/UX wireframes and interface layouts.
- **Docker:** Used for containerized application deployment and environment management.

These tools supported the development workflow and helped the team manage, test, and maintain the PolicyGPT application.

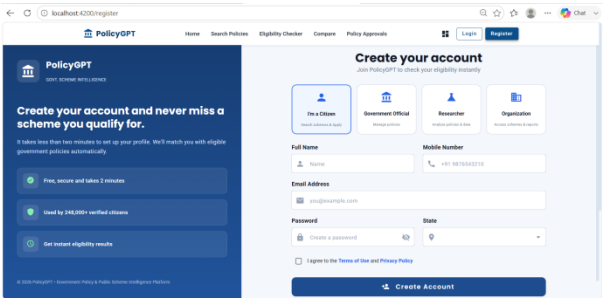
CHAPTER 6
MODULES AND IMPLEMENTATION

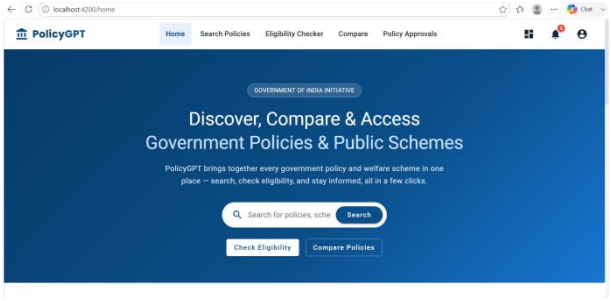
6.1 User Authentication and Role Management

The **User Authentication and Role Management Module** provides secure access to the PolicyGPT platform and ensures that users can access functionalities according to their assigned roles. The module supports **user registration, secure login, JWT-based authentication, password reset, profile management, and role-based access control**.

6.1.1 User Registration and Login

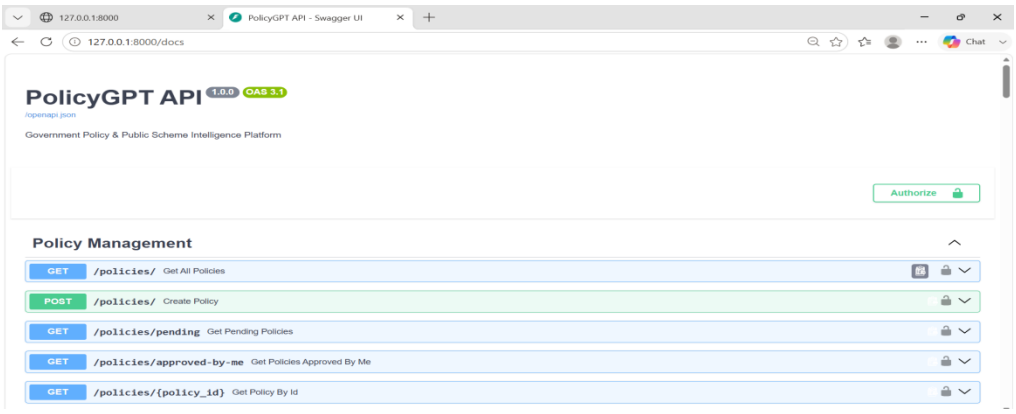
New users can register by providing the required details and can later log in using their credentials. After successful authentication, users are given access according to their assigned role.





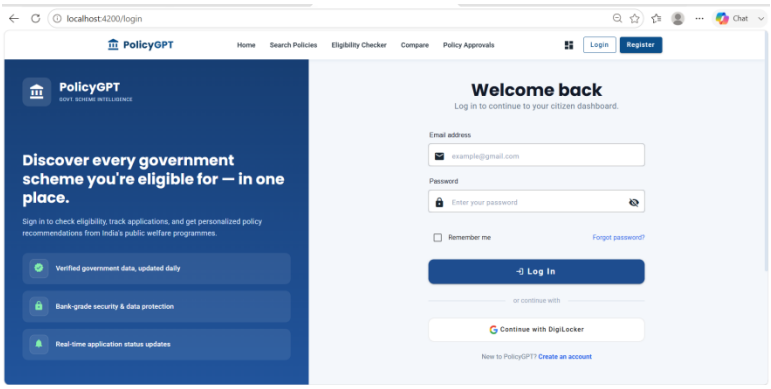
6.1.2 JWT Authentication and Access Control

The system uses **JWT-based authentication** to secure protected resources. Role-based access control ensures that users can access only the features permitted for their role.



6.1.3 Password and Profile Management

Users can manage their account information and password through the available profile and password-management features.



6.2 Policy and Public Scheme Management Module

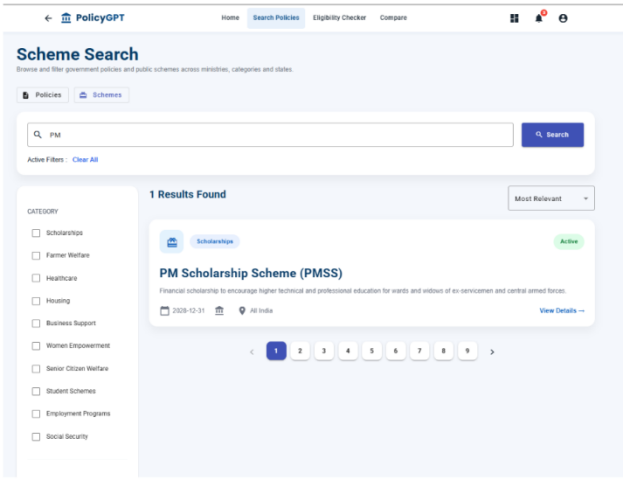
The **Policy and Public Scheme Management Module** manages government policies and public welfare schemes within the PolicyGPT platform. It allows authorized users to add, organize, update, and maintain policy and scheme information.

6.2.1 Policy Management

The policy management functionality supports:

- Policy upload and registration
- Policy categorization
- Policy editing and updates
- Policy approval
- Policy search
- Policy archiving

Policies can be organized into categories such as **Education, Healthcare, Agriculture, Employment, Finance, Housing, Environment, and Digital Governance**.



6.2.2 Public Scheme Management

The public scheme management functionality maintains information about government welfare schemes. It supports scheme registration, details management, categorization, eligibility rules, updates, and archiving.

Scheme categories include **Scholarships, Farmer Welfare, Healthcare, Housing, Business Support, Women Empowerment, Student Schemes, Employment Programs, and Social Security**.

6.2.3 Module Workflow

The basic workflow is:

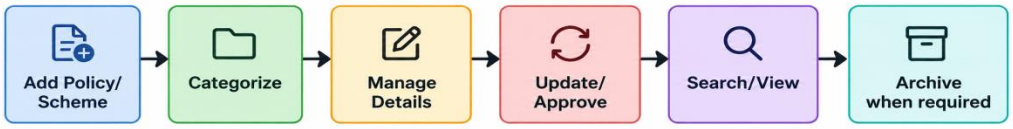


Figure 6.2: Workflow of Policy and Public Scheme Management Module

This module provides the foundation for the **Search, Eligibility, Comparison, Notification, and Reporting** features of PolicyGPT.

6.3 Policy Search, Eligibility Checker and Policy Comparison Modules

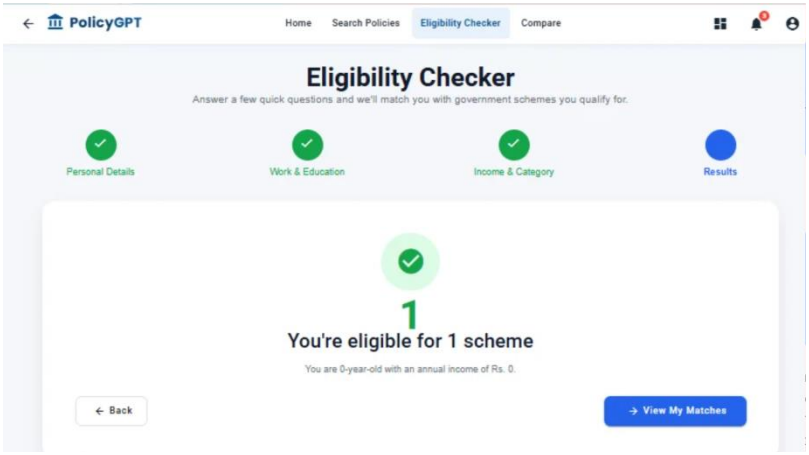
This section combines three related modules that help users **find relevant policies and schemes, check scheme eligibility, and compare available options**.

6.3.1 Policy Search

The Policy Search Module allows users to find policies and schemes using **keywords and filters**. Users can search by category, state, ministry, department, policy name, scheme name, sector, publication date, and status.

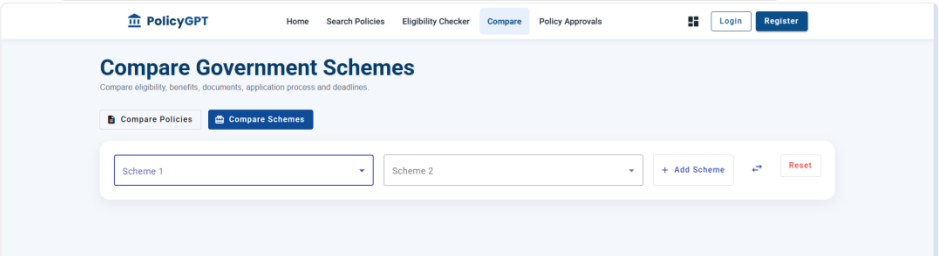
6.3.2 Eligibility Checker

The Eligibility Checker helps users identify schemes that may match their profile. Users provide details such as **age, gender, income, occupation, education, location, social category, and disability status**. The system performs profile-based scheme matching and provides an eligibility summary, recommended schemes, and application guidance.



6.3.3 Policy Comparison

The Policy Comparison Module allows users to compare policies and government schemes based on important aspects such as **benefits, eligibility, departments, and application processes**. This helps users understand the differences between available options before making an informed choice.

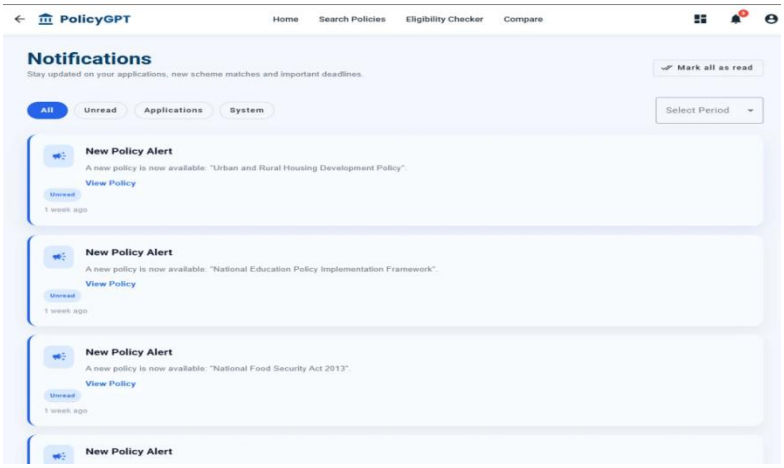


6.4 Notifications and Application Tracking

The **Notifications and Application Tracking Module** keeps users informed about important policy, scheme, deadline, and application-related updates. It supports **new policy alerts, scheme update notifications, deadline reminders, and application notifications** through available notification channels.

6.4.1 Policy and Scheme Notifications

Users can receive notifications when new policies are added or when existing public schemes are updated. This helps users stay informed about relevant changes.



6.4.2 Deadline and Application Notifications

The module provides **deadline reminders** and **application notifications** to help users track important application-related activities and avoid missing significant updates.

6.4.3 Notification Channels

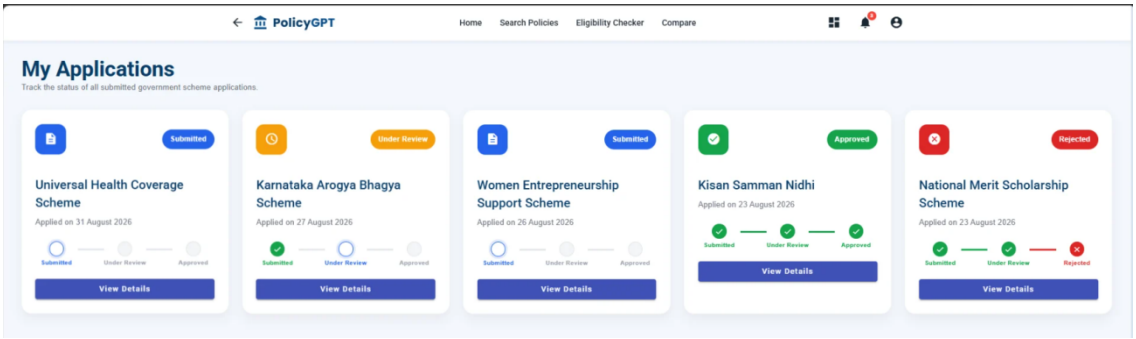
PolicyGPT supports multiple notification channels:

- **Email Notifications**
- **SMS Notifications**
- **In-App Notifications**

These channels provide users with timely information based on the type of notification and available services.

6.5 Dashboard, Analytics and Reports Modules

The **Dashboard, Analytics and Reports Modules** provide users with relevant information, statistics, and reports according to their access level. Dashboards help users monitor policies, schemes, notifications, activities, and application-related information.



6.5.1 Dashboards

PolicyGPT provides role-based dashboards with different information for different users:

- **Citizen Dashboard:** Saved policies, eligible schemes, recent notifications, search history, and application status.
- **Government Dashboard:** Policy statistics, scheme usage analytics, user activity, department reports, and notification statistics.
- **Admin Dashboard:** User management, policy management, analytics, reports, and audit logs.

6.5.2 Analytics

The analytics functionality helps authorized users understand **policy statistics, scheme usage, user activity, and notification-related information**. This supports better monitoring and information management.

6.5.3 Reports and Export

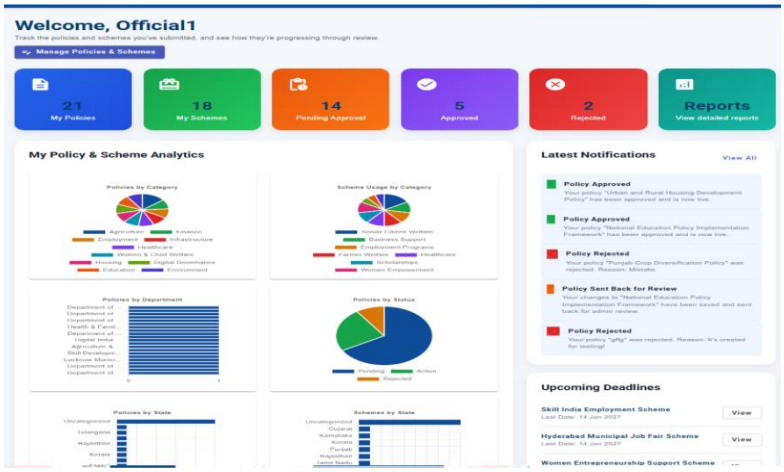
The Reports Module supports generation of:

- Policy Reports
- Scheme Reports
- User Activity Reports
- Department Reports
- Analytics Reports

Reports can also be exported in **PDF and Excel formats**.

6.5.4 Overall Outcome

The dashboards and reporting features provide a centralized view of important system information and help users and administrators make better-informed decisions.



6.6 Feedback and Support

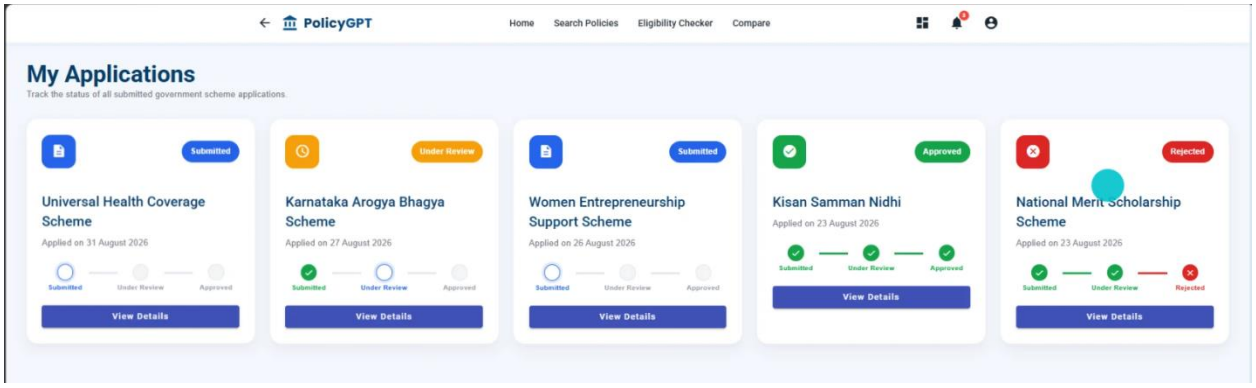
The **Feedback and Support Module** allows users to provide feedback, report issues, and obtain assistance while using the PolicyGPT platform. It helps improve user interaction and supports the resolution of user queries.

The module includes the following features:

- **Citizen Feedback** – Allows users to submit feedback about the platform.

- **Issue Reporting** – Enables users to report problems or issues.
- **Help Desk** – Provides assistance for user queries.
- **FAQ Management** – Provides answers to frequently asked questions.
- **Query Resolution** – Supports handling and resolving user queries.
- **Contact Support** – Provides a channel for contacting support.

Overall, the module improves user support and provides a mechanism for collecting feedback and addressing issues within the platform.



CHAPTER 7
DATABASE DESIGN AND IMPLEMENTATION

7.1 Database Overview and Architecture

PolicyGPT uses **PostgreSQL** as its primary database for storing and managing application data. The database is designed to support major system modules including users, policies, public schemes, eligibility rules, notifications, feedback, reports, audit logs, and search history.

The database follows a structured approach in which related information is maintained in separate tables and connected through appropriate relationships. This supports organized data management and allows the different modules of PolicyGPT to access the required information.

7.1.1 Main Database Entities

The major entities considered in the database design are:

- **Users** – Stores user and role-related information.
- **Policies** – Stores government policy details.
- **Schemes** – Stores public welfare scheme information.
- **Eligibility Rules** – Maintains eligibility criteria for schemes.
- **Notifications** – Stores user notification information.
- **Feedback** – Maintains feedback and issue-related information.
- **Reports** – Stores report-related records.
- **Audit Logs** – Maintains records of relevant system activities.
- **Search History** – Stores user search-related information.

The database design provides the required data foundation for authentication, policy and scheme management, eligibility checking, notifications, dashboards, reporting, and other modules of the platform.

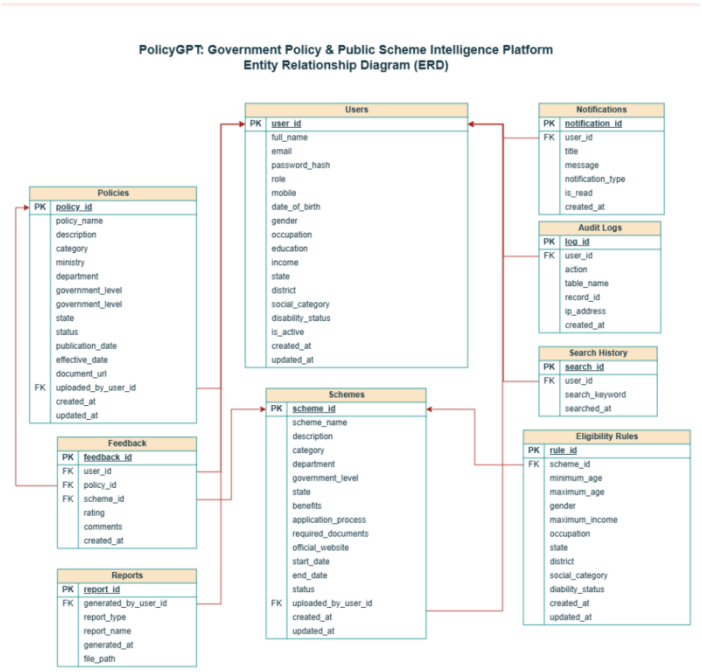
7.2 Entity Relationship Diagram

The **Entity Relationship (ER) Diagram** represents the logical structure of the PolicyGPT database and shows the relationships between the major entities. It provides a visual representation of how information related to users, policies,

schemes, eligibility, notifications, feedback, reports, and system activities is organized.

The major entities identified in the database design include **Users, Policies, Schemes, Eligibility Rules, Notifications, Feedback, Reports, Audit Logs, and Search History**.

The ER diagram helps in understanding the relationships between these entities and provides a foundation for implementing the database structure required by the different PolicyGPT modules.



7.3 Database Tables and Relationships

The PolicyGPT database consists of separate tables for managing user information, policies, schemes, eligibility, notifications, feedback, reports, and system activities. The main tables identified in the project database design are **users, policies, schemes, eligibility_rules, notifications, feedback, reports, audit_logs, and search_history**

7.3.1 Main Database Tables

S. No.	Table	Purpose
1	**users**	Stores user account and role information.
2	**policies**	Stores government policy details and related information.
3	**schemes**	Stores public welfare scheme details.
4	**eligibility_rules**	Stores eligibility criteria associated with schemes.
5	**notifications**	Stores notifications and alerts for users.
6	**feedback**	Stores user feedback and reported issues.
7	**reports**	Maintains generated report information.
8	**audit_logs**	Records relevant system activities for monitoring.
9	**search_history**	Stores user search-related information.

7.3.2 Table Relationships

The tables are logically connected to support the different modules of PolicyGPT. User information is associated with activities such as **notifications, feedback, search history, reports, and audit records**. Policies and schemes contain the information used by the **search, eligibility, comparison, notification, and reporting modules**. Eligibility rules are associated with schemes to support eligibility checking.

The database structure therefore provides a common data layer for the major functionalities of the PolicyGPT platform.

CHAPTER 8
TESTING AND VALIDATION

8.1 Testing Strategy

Testing was performed to verify that the different modules of **PolicyGPT** function correctly and meet the expected requirements. The testing process focused on **module functionality, user inputs, API responses, and the overall application workflow**.

The testing strategy covered the following areas:

1. **Functional Testing** – Verified that each module performs its intended functionality.
2. **Unit Testing** – Tested individual components and functions.
3. **Integration Testing** – Verified communication between the frontend, backend, and database.
4. **API Testing** – Checked backend API requests and responses.
5. **User Interface Testing** – Verified that application interfaces and user interactions work correctly.

Testing was carried out across the major modules, including **authentication, policy and scheme management, search, eligibility checking, comparison, notifications, dashboards, reports, and feedback**. The identified issues were addressed during the testing and fixing phase

8.2 Functional, Unit and Integration Testing

Testing was carried out at different levels to ensure that individual components and integrated modules of PolicyGPT work as expected.

8.2.1 Unit Testing

Unit testing focused on individual functions and components. It helped verify that specific operations performed correctly before being integrated with other parts of the application.

8.2.2 Functional Testing

Functional testing verified whether each module performed its intended functionality according to the project requirements. Major areas tested included authentication, policy and scheme management, search, eligibility checking, comparison, notifications, dashboards, reports, and feedback.

8.2.3 Integration Testing

Integration testing verified the interaction between the **Angular frontend, FastAPI backend, and PostgreSQL database**. It ensured that requests, responses, and data flow between the different application layers worked correctly.

Overall, these testing levels helped identify functional and integration issues before the final application demonstration.

8.3 API and User Interface Testing

8.3.1 API Testing

API testing was performed to verify the communication between the **Angular frontend and FastAPI backend**. API requests and responses were checked to ensure that the backend services return the expected results for different operations. **Postman** was used for testing backend API requests and responses.

8.3.2 User Interface Testing

User Interface testing was performed to ensure that the application screens, input fields, buttons, navigation, and user interactions work correctly. The interfaces were checked for proper interaction and consistency across the major modules.

8.3.3 Validation

The testing process also verified user inputs and the overall application workflow to ensure that information moves correctly between the frontend, backend, and database.

8.4 Test Cases and Results

Test cases were prepared to validate the major PolicyGPT modules and verify that they functioned as expected.

S. No.	Module	Test Scenario	Expected Result	Result
1	User Authentication	Register and login with valid credentials	User should be registered and authenticated successfully	Pass
2	Policy & Scheme Management	Add or update policy/scheme details	Information should be stored and displayed correctly	Pass
3	Policy Search	Search using keywords and apply filters	Relevant filtered results should be displayed	Pass
4	Eligibility Checker	Submit valid user eligibility details	Suitable eligible schemes should be identified	Pass
5	Policy Comparison	Select policies/schemes for comparison	Relevant details should be displayed for comparison	Pass
6	Notifications	Trigger a policy, scheme, deadline, or application update	Appropriate notification should be generated	Pass
7	Dashboard & Analytics	Access dashboard and view statistics	Relevant information and statistics should be displayed	Pass
8	Reports	Generate a policy, scheme, or analytics report	Requested report should be generated successfully	Pass
9	Feedback & Support	Submit feedback or report an issue	Feedback/issue should be recorded successfully	Pass
10	Access Control	Attempt to access a restricted feature	Unauthorized access should be restricted	Pass

8.4.1 Testing Outcome

The test results confirmed that the major modules performed their intended functions. Testing also helped identify and resolve issues related to inputs, API communication, integration, and user interactions.

8.5 Error Handling and Issue Fixing

During development and testing, errors related to user inputs, API communication, module integration, and system functionality were identified and resolved. Appropriate validation and error-handling mechanisms were

applied to improve system reliability and ensure smooth interaction between the frontend, backend, and database.

8.5.1 Error Handling

- **Input Validation:** Invalid or incomplete user inputs are validated before processing.
- **API Error Handling:** Backend errors and unsuccessful API responses are handled appropriately.
- **Authentication Errors:** Invalid login credentials and unauthorized access are restricted.
- **Integration Errors:** Communication issues between Angular, FastAPI, and PostgreSQL were identified and fixed.
- **User Feedback:** Appropriate messages are provided when an operation fails or requires correction.

8.5.2 Issue Fixing

Identified issues were analyzed during testing and corrected through debugging and validation. The fixes improved module integration, data handling, API communication, and overall application reliability.

CHAPTER 9
MILESTONE-WISE DEVELOPMENT

9.1 Milestone 1 – Requirements, UI and Database Design

Duration: Week 1–2

Milestone 1 focused on establishing the foundation of the PolicyGPT platform through requirements analysis, UI design, database design, and backend/frontend setup.

Key Activities

- Defined project objectives and functional/non-functional requirements.
- Designed UI wireframes and user flow diagrams.
- Prepared dashboard layouts and responsive screens using Figma.
- Designed the database schema.
- Initialized the FastAPI backend and Angular frontend.
- Configured PostgreSQL.
- Implemented JWT authentication and configured Angular Material.

Major Outcomes

- UI wireframes completed.
- User flows finalized.
- Database schema completed.
- FastAPI backend initialized.
- Angular frontend initialized.
- Authentication implemented.

9.2 Milestone 2 – Policy, Scheme, Search and Eligibility

Duration: Week 3–4

Milestone 2 focused on developing the core information management and user-oriented functionalities of PolicyGPT.

Key Activities

- Developed the Policy Management module.
- Built the Public Scheme Management module.
- Implemented policy and scheme search functionality.
- Created the Policy Approval workflow.
- Developed the Eligibility Checker.
- Developed the Policy Comparison module.

Major Outcomes

- Policy repository became operational.
- Scheme management functionality was completed.
- Search functionality became operational.
- Eligibility Checker became functional.

9.3 Milestone 3 – Analytics, Notifications and Reporting

Duration: Week 5–6

Milestone 3 focused on extending the platform with analytics, notifications, reporting, feedback, and usage monitoring capabilities.

Key Activities

- Developed the Analytics Dashboard.
- Built the Notification System.
- Generated policy, scheme, and analytics reports.
- Implemented the Feedback module.
- Developed Department Analytics.
- Built the Usage Statistics Dashboard.

Major Outcomes

- Analytics dashboard completed.
- Notification system operational.
- Reports generated successfully.
- Feedback management completed.

9.4 Milestone 4 – Testing, Deployment and Documentation

Duration: Week 7–8

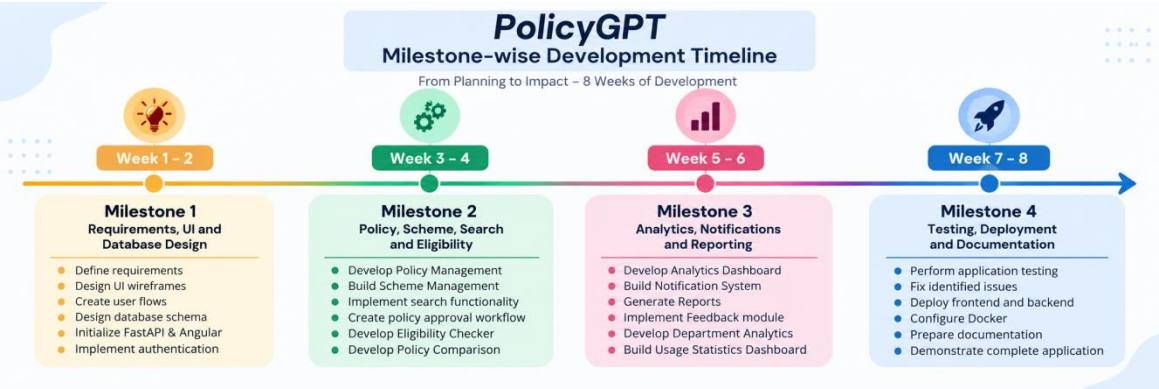
Milestone 4 focused on validating the complete application, resolving identified issues, preparing deployment, and completing project documentation.

Key Activities

- Performed application testing.
- Fixed identified issues.
- Deployed the frontend and backend.
- Configured Docker.
- Prepared project documentation.
- Demonstrated the complete application.

Major Outcomes

- Production-ready application prepared.
- Deployment completed.
- Documentation completed.
- Complete project demonstration successfully conducted.



CHAPTER 10
DEPLOYMENT and RESULTS

10.1 Deployment Overview and Configuration

PolicyGPT was developed as a full-stack application using Angular, FastAPI, and PostgreSQL. Docker was used for application setup and deployment. The frontend communicates with the backend through APIs.

10.2 Module-wise Results and Overall Outcome

The developed PolicyGPT platform successfully integrates the major functional modules into a centralized government policy and public scheme information platform.

Module	Result
User Authentication	Secure authentication and role-based access implemented
Policy Management	Policy information can be managed and organized
Scheme Management	Public scheme information can be maintained
Policy Search	Keyword search and filtering functionality implemented
Eligibility Checker	User eligibility can be evaluated for suitable schemes
Policy Comparison	Policies and schemes can be compared
Notifications	Policy, scheme, deadline, and application notifications supported
Dashboard & Analytics	Relevant statistics and activities can be viewed
Reports & Export	Policy, scheme, and analytics reports supported
Feedback & Support	Feedback and issue reporting functionality provided

CHAPTER 11
CONCLUSION AND FUTURE ENHANCEMENTS

11.1 Conclusion and Project Contributions

PolicyGPT successfully provides a centralized platform for accessing, searching, and managing government policies and public welfare schemes. The system integrates policy management, scheme management, search, eligibility checking, comparison, notifications, analytics, reporting, and feedback features. The project improves the accessibility and organization of government information for different categories of users.

11.2 Future Enhancements

Future enhancements may include:

- AI-powered conversational policy assistance.
- Multilingual support for regional languages.
- Advanced personalized scheme recommendations.
- Mobile application integration.
- Real-time government policy updates.
- Enhanced analytics and visualization.
- Integration with additional government data sources.